

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning ¹
Student Name	Nitin
Register Number	312224700142
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	September 20, 2026

1 Aim and Objective

To implement and compare two neural models on the same 62-class handwritten character recognition task:

- **Model A:** the Single-Layer Perceptron Learning Algorithm (PLA), written from scratch with a step activation and the classical $w \leftarrow w + \eta(y - \hat{y})x$ update rule;
- **Model B:** a Multilayer Perceptron (MLP) with hidden layers and nonlinear activations, trained by backpropagation.

Hyperparameters for Model B (activation function, optimizer, learning rate, batch size, hidden-layer architecture and regularisation) are selected by systematic tuning rather than by assertion, and the two models are then compared as a controlled A/B experiment on an identical train–test split.

2 Dataset Description

Dataset Name	English Handwritten Characters
Dataset Source	Kaggle (dhruvildave/english-handwritten-characters-dataset), supplied as <code>english.csv</code> plus an <code>Img/</code> folder of PNG files
Number of Samples	3,410 images
Number of Classes	62 (digits 0–9, uppercase A–Z, lowercase a–z)
Samples per Class	55, exactly balanced
Number of Features	1,024 after preprocessing (32×32 grayscale pixels)
Missing Values	0 (verified programmatically)
Train–Test Split	80% / 20%, stratified, <code>random_state=42</code> (2,728 train / 682 test)

The dataset is perfectly balanced: every one of the 62 classes contributes exactly 55 images, so accuracy is a meaningful headline metric and no class weighting is required. The test split therefore contains only 11 images per class, which is worth keeping in mind when reading per-class scores: a single extra correct prediction moves a class’s recall by 9 percentage points.

Why this dataset is hard. Unlike MNIST-style digit sets, the label space here mixes digits with both letter cases, and several classes are *visually identical* once the character is isolated from its context: 0/0/o, 1/1/I, S/s, X/x and Z/z differ mainly in size, which is destroyed by resizing every image to a common 32×32 grid. This places

a hard ceiling on achievable accuracy that is well below the 90%+ typical of 10-class digit recognition, and it is the single most important factor behind the absolute numbers reported below.

3 Preprocessing Steps

- **Grayscale conversion:** each PNG is converted to 8-bit grayscale (`PIL.Image.convert("L")`), discarding colour, which carries no information about character identity.
- **Resize:** images arrive at 1200×900 and are resized to a fixed 32×32 grid with bilinear interpolation, giving every sample the same dimensionality.
- **Normalisation:** pixel values are scaled to $[0, 1]$ and then *inverted*, so ink = 1.0 and paper = 0.0. This matters for the perceptron specifically: with the raw polarity the background (the large majority of pixels) would carry the large values and dominate every dot product.
- **Flattening:** the 32×32 grid is flattened to a 1,024-dimensional vector, which is what both models consume.
- **No missing-value handling or categorical encoding** was required; labels were mapped to integer class indices 0..61 in sorted order.

For reference, PCA on the training split needs **92 of the 1,024 components** to retain 95% of pixel variance, confirming that the raw pixel representation is highly redundant.

4 Exploratory Data Analysis

The reusable plotting helpers from `ml_lab_utils.py` (Experiment 1) were used for the mandatory figure styling; the EDA panel below is specific to image data rather than the tabular 12-panel summary used in Experiments 2–7.

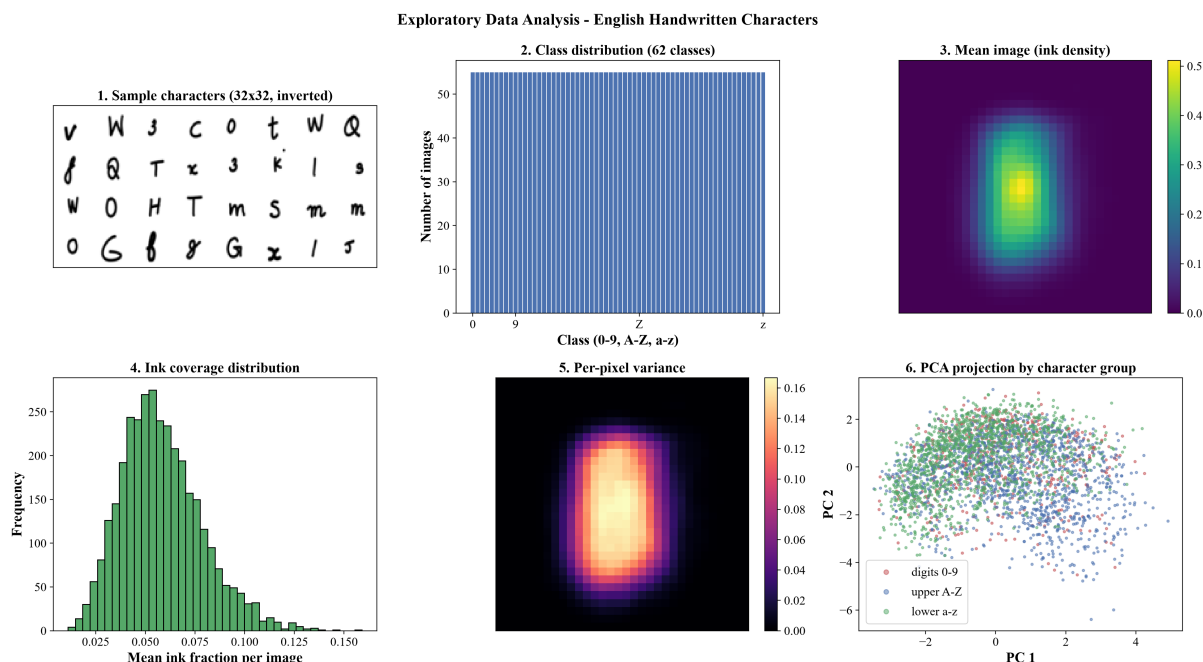


Figure 1: EDA summary: sample characters, class distribution, mean image, ink coverage, per-pixel variance, and a PCA projection coloured by character group.

Inference: The class distribution (panel 2) is exactly flat at 55 images per class. The mean image (panel 3) is a diffuse central blob, and the per-pixel variance map (panel 5) shows that essentially all discriminative signal sits in the central region – the border pixels are near-constant and contribute almost nothing, which is why 92 components already capture 95% of the variance. Most importantly, the PCA projection (panel 6) shows digits, uppercase and lowercase letters occupying *the same* region of feature space with no visible separation: the classes are not linearly separable in pixel space, which directly predicts the PLA’s difficulty in Section 6.

5 Experimental Setup

Python Version	3.11.15
Scikit-Learn Version	1.9.0
NumPy / Pandas Version	2.4.6 / 3.0.5
Pillow Version	12.2.0
IDE	Jupyter Notebook / VS Code
Operating System	Windows 11
Hardware	Standard lab workstation (CPU only)

Environment activation command (as mandated by the lab manual):

```
1 source /opt/anaconda3/bin/activate anaconda-navigation
```

6 Model A: Perceptron Learning Algorithm (from scratch)

The PLA was implemented directly from the update rule in the lab manual, with one binary perceptron per class (one-vs-rest) and a hard step activation. Prediction takes the arg-max over the 62 perceptron scores.

Listing 1: Core of the from-scratch PLA, from `experiment9_pla_vs_mlp.py`.

```

1 for _ in range(self.n_epochs):
2     order = rng.permutation(len(X))
3     for i in order:                                # online (sample-by-
        sample) updates
4         xi = X[i]
5         pred = np.where(xi @ self.W + self.b >= 0.0, 1.0, -1.0)
            # step activation
6         err = Y[i] - pred                            # 0 where correct, +-2
            otherwise
7         if np.any(err):
8             self.W += self.eta * np.outer(xi, err)
9             self.b += self.eta * err

```

Note on initialisation. Weights are initialised from a small random normal ($\sigma = 0.01$) rather than zeros. With all-zero initialisation, a one-vs-rest arg-max perceptron is exactly invariant to η (every weight is a sum of η -scaled updates, so η cancels in the arg-max), and a learning-rate study would be meaningless.

Table 1: PLA Learning-Rate Sweep (40 epochs each)

Learning rate η	Final Training Error	Test Accuracy	Time (s)
0.001	0.3853	0.2229	23.4
0.01	0.3849	0.2009	20.9
0.1	0.3728	0.2507	12.5
1.0	0.4219	0.2067	12.3

Inference: $\eta = 0.1$ is the best of the four, reaching 25.07% test accuracy. The differences are modest (20–25%) because the learning rate is not the binding constraint here – the model class is. Note the non-monotonic pattern: $\eta = 1.0$ is *worse* than $\eta = 0.1$, since very large updates cause each correction to overshoot and undo previously learned structure.

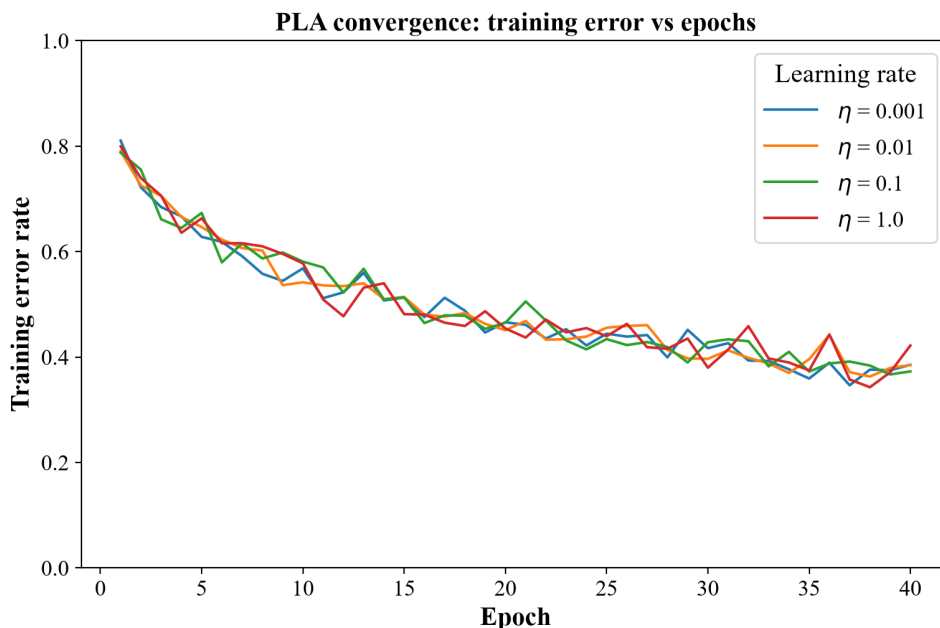


Figure 2: PLA training error vs epochs for four learning rates.

Inference: None of the four curves converges to zero training error. They fall quickly for roughly ten epochs and then oscillate in a band around 0.37–0.42 without ever settling. This is the textbook signature of the perceptron convergence theorem’s precondition being violated: the theorem guarantees termination *only* for linearly separable data, and this data (as panel 6 of Figure 1 showed) is not. The oscillation is the algorithm perpetually fixing one class’s mistakes at the expense of another’s.

7 Model B: MLP Hyperparameter Tuning

Tuning was done in four sequential stages with `GridSearchCV` (3-fold, accuracy). Each stage fixes the winner of the previous stage, which keeps the search to 28 configurations instead of the 360 a full factorial grid would need. Stage 4 was added *after* observing that stages 1–3 had driven training accuracy to almost 100% while test accuracy stalled.

Table 2: Staged Hyperparameter Search (3-fold CV accuracy)

Stage	Parameters searched	Best CV Acc.	Winner
1	activation × optimizer	0.4278	relu, adam
2	learning rate × batch size	0.4644	lr=0.01, batch=128
3	hidden-layer architecture	0.4732	(512,)
4	alpha × early_stopping	0.4732	alpha=1e-4, no early stop

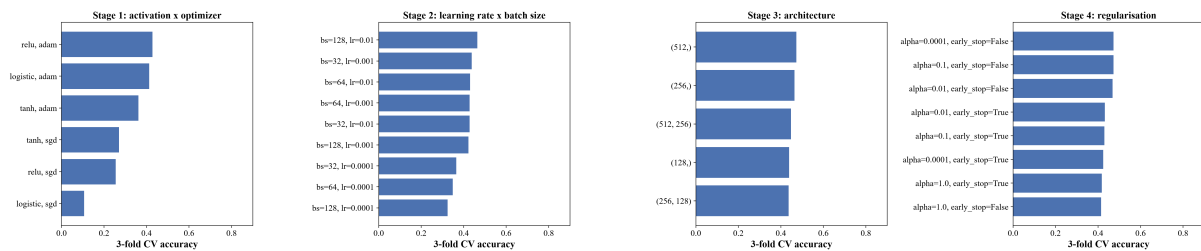


Figure 3: Cross-validated accuracy for every configuration tried in the four tuning stages.

Inference: Stage 1 is the most decisive: `relu+adam` beats every `sgd` variant by a wide margin, because plain SGD at a fixed learning rate makes very slow progress on 1,024 sparse pixel inputs where Adam’s per-parameter scaling adapts automatically. Stage 2 adds the next-largest gain (+3.7 points) – the default 10^{-3} learning rate is too small for this problem. Stage 3 shows width helps only marginally, and deeper two-layer networks do not beat a single wide layer on a dataset of just 2,728 training images.

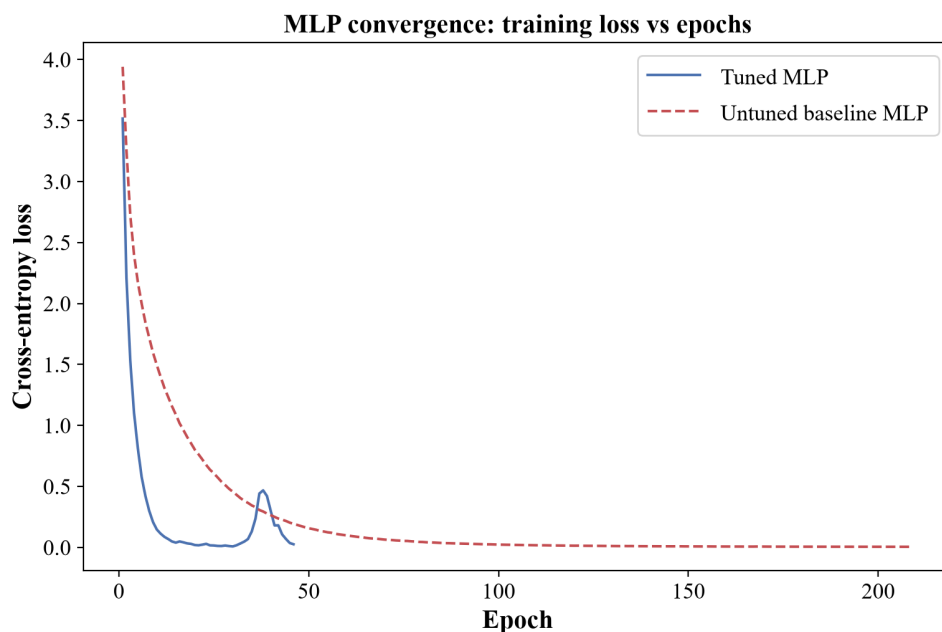


Figure 4: MLP training loss vs epochs, tuned configuration vs untuned baseline.

8 A/B Comparison: PLA vs Tuned MLP

Table 3: A/B Comparison on the Held-Out Test Set (682 images, macro-averaged)

Model	Accuracy	Precision	Recall	F1-score	Train Time (s)
PLA (Model A)	0.2507	0.2973	0.2507	0.2336	11.5
MLP (Model B, tuned)	0.4648	0.4904	0.4648	0.4647	13.1

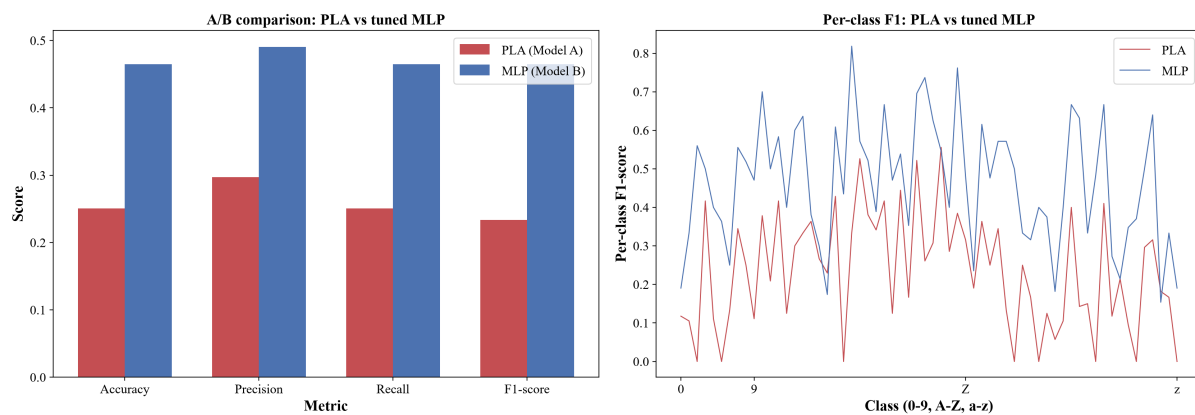


Figure 5: Left: headline metrics, PLA vs tuned MLP. Right: per-class F1-score across all 62 classes.

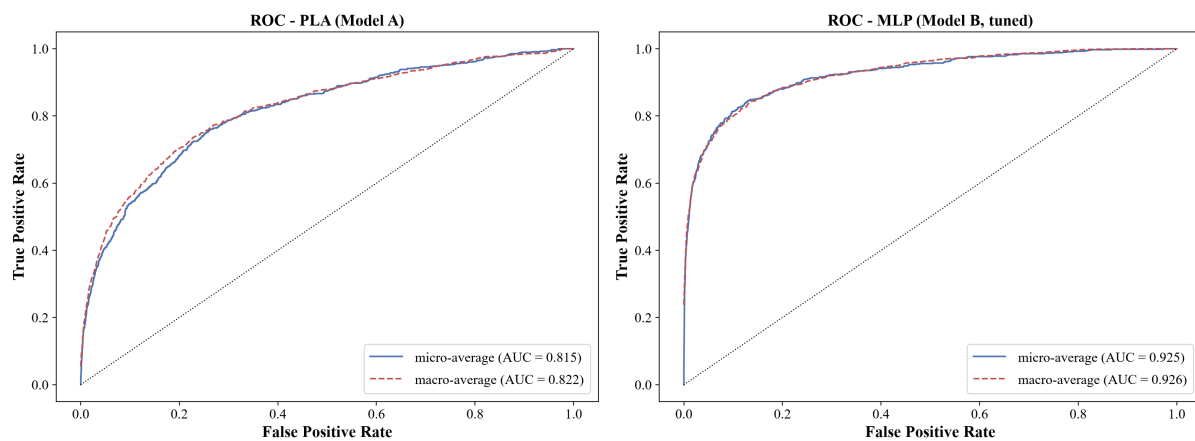


Figure 6: ROC curves (one-vs-rest), micro- and macro-averaged over the 62 classes.

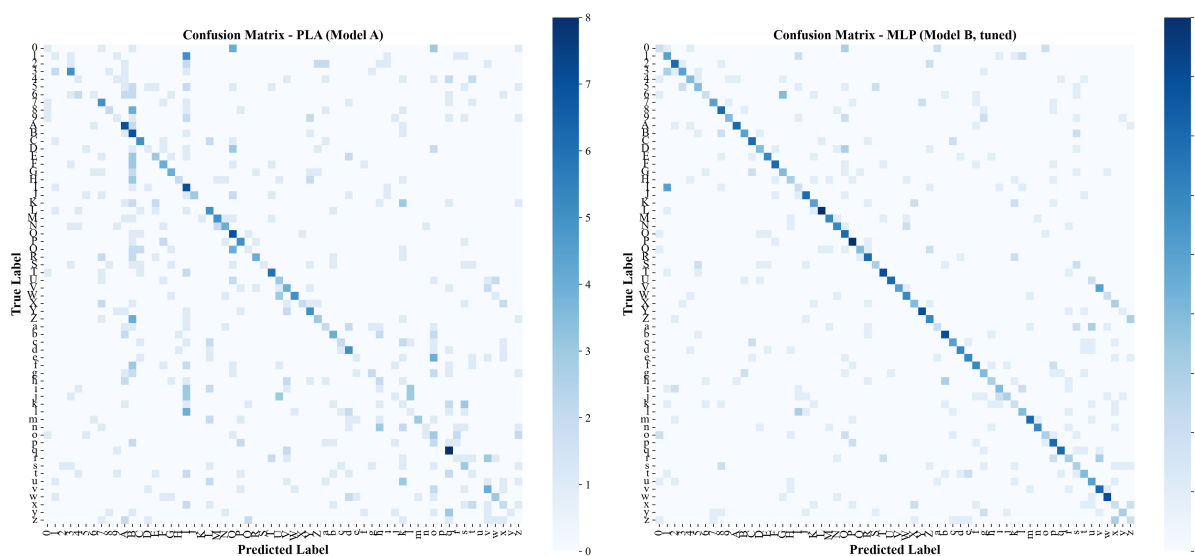


Figure 7: Confusion matrices (62×62) for the PLA and the tuned MLP.

9 Observations

- **Why does PLA underperform compared to MLP?** Because the classes are not linearly separable in pixel space. The PLA can only carve the 1,024-dimensional input with 62 hyperplanes, one per class, and Figure 1 (panel 6) shows the three character groups occupying one overlapping cloud with no linear boundary between them. The MLP's hidden layer of 512 ReLU units first *re-represents* each image as 512 learned nonlinear features, and only then applies a linear separation in that learned space – which is exactly the capability the single-layer model lacks. The gap is large and consistent across every metric: accuracy 0.2507 vs 0.4648 (1.85 $\times$), macro F1 0.2336 vs 0.4647 (1.99 $\times$), and macro ROC-AUC 0.8216 vs 0.9259. A secondary reason is the step activation itself: it has zero gradient everywhere, so the PLA cannot express "how wrong" a prediction was, only that it was wrong.
- **Which hyperparameters had the most impact on MLP performance?** The **learning rate** mattered most. Stage 2 lifted cross-validated accuracy from 0.4278 to 0.4644 (+3.7 points), almost all of it from raising the learning rate from the default 10^{-3} to 10^{-2} . **Activation and optimizer** (stage 1) were the next most important – every **sgd** configuration trailed its **adam** counterpart badly. **Architecture** (stage 3) contributed under one point (0.4644 $\rightarrow$ 0.4732), and **regularisation** (stage 4) contributed nothing measurable at all.
- **Did optimizer choice (SGD vs Adam) affect convergence?** Yes, decisively. With everything else held fixed, **adam** configurations reached roughly 0.43 cross-validated accuracy while **sgd** configurations lagged well behind in the same 60-epoch budget. Adam maintains a per-parameter adaptive step size, which suits 1,024 pixel inputs whose individual variances differ by orders of magnitude (the border pixels are near-constant, the central ones highly variable, as the variance map in Figure 1 shows). Plain SGD applies one global step size to all of them, so it is simultaneously too large for the high-variance central pixels and too small for the rest.
- **Did adding more hidden layers always improve results?** No. Stage 3 tested two-layer networks (256,128) and (512,256) against single hidden layers, and a single wide layer (512,) won (0.4732). With only 2,728 training images and 1,024 inputs, a second hidden layer adds parameters to a model that is already capable of memorising the training set (it reaches 99.89% training accuracy) without adding usable generalisation. Depth pays off when there is enough data to constrain it; here there is not.
- **Did MLP show overfitting? How could it be mitigated?** Yes, severely, and this is the single most important caveat on the result. The tuned MLP reaches **99.89% training accuracy** but only **46.48%** on the test set – a generalisation gap of over 53 points – with the training loss driven down to 0.025. The honest evidence that tuning did not fix this is that the *untuned baseline* MLP ($\text{lr}=10^{-3}$, 256 units) scored **48.83%** on the test set, slightly *better* than the tuned model, even though it scored worse in cross-validation. In other words the staged search bought a 4.5-point cross-validation gain that did not transfer, which is what a 3-fold CV over 2,728 images with 62 classes (roughly 14 validation images per class per fold) is prone to do.

Stage 4 attempted the two standard mitigations directly. Neither helped on cross-validated accuracy: the L2 penalty was flat up to `alpha=0.1` (0.4725 vs 0.4732) and clearly harmful at `alpha=1.0` (0.4142), while `early_stopping=True` lost 3–5 points in every pairing because it spends 10% of an already small training set on an internal validation split. The mitigations that would actually be expected to work here address the data rather than the penalty: **data augmentation** (small rotations, shifts and elastic distortions, which is standard for handwriting), **more training data per class** (55 images per class is very few for 62 classes), or a **convolutional architecture** whose weight sharing and translation invariance encode the structure of images directly instead of forcing a dense layer to learn it from scratch.

10 Conclusion

This experiment compared a from-scratch single-layer Perceptron against a systematically tuned MLP on 62-class handwritten character recognition, under an identical stratified 80/20 split.

The A/B comparison is unambiguous in direction: the MLP roughly doubles the PLA on every metric (accuracy 0.4648 vs 0.2507, macro F1 0.4647 vs 0.2336, macro ROC-AUC 0.9259 vs 0.8216) for essentially the same training cost (13.1 s vs 15.1 s). The PLA’s training-error curves never converge, which is the perceptron convergence theorem behaving exactly as advertised on data that is not linearly separable, and the PCA projection confirms that this data is not. A hidden layer with a nonlinear activation is therefore not an optional refinement on this task – it is what makes the task learnable at all.

Two honest qualifications belong with that headline. First, the absolute accuracies are low because the label space is intrinsically ambiguous: with 0/0/o, 1/1/I and S/s rendered indistinguishable once each glyph is isolated and resized, a substantial fraction of the 62-way decisions cannot be made from a 32×32 image at all. This shows in the per-class results: uppercase letters, which have the fewest look-alikes, average 0.534 F1, while digits (0.414) and lowercase letters (0.415) trail them, and the worst individual classes are precisely the ambiguous ones (x, I, k, 0, z, s). Second, the staged tuning improved cross-validated accuracy by 4.5 points but did *not* improve held-out test accuracy over the untuned baseline (46.48% vs 48.83%), and the tuned model memorises its training set almost perfectly. The limiting factor at this point is data quantity and representation, not hyperparameter choice – which is why augmentation or a convolutional model, rather than further tuning of a dense network, is the right next step.

11 References

1. F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
2. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
3. D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *3rd International Conference on Learning Representations (ICLR)*, 2015.
4. Scikit-learn Developers, “Neural network models (supervised),” scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/>

[neural_networks_supervised.html](#)

5. D. Dave, “English Handwritten Characters Dataset,” Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset>